

Objects & Array Methods

grouping data, and the methods that work on it

1 What is an object?

An **object** is a collection of related information stored as **key-value pairs**. Think of a student's admission form: name, age, city, marks, all kept together in one place.

```
const student = {  
  name: "Rahul",  
  age: 16,  
  city: "Delhi",  
  marks: 92  
};
```

Each line is a **key** (name, age...) paired with a **value** ("Rahul", 16...).

2 Why objects?

- group related data in one place
- readable code - the keys explain what each value is
- reusable and easy to maintain
- used everywhere: users, products, orders, employees

3 Accessing properties

Two ways to read a value out of an object:

```
student.name;           // dot notation  
student["age"];         // bracket notation
```

When to use which: dot notation is the everyday choice. Use brackets when the key is stored in a variable, or has a space or special character in it.

4 Add, update & delete

```
student.school = "ABC School"; // add
student.marks = 95;           // update
delete student.city;          // delete
```

5 Methods and this

A **method** is a function stored inside an object. Inside it, **this** refers to the object itself, so it can reach its own data.

```
const student = {
  name: "Rahul",
  greet() {
    console.log("Hello " + this.name);
  }
};
student.greet(); // Hello Rahul
```

6 Looping through an object

```
// visit every key
for (const key in student) {
  console.log(key, student[key]);
}
```

Three helpers that turn an object into arrays you can work with:

method	gives you
Object.keys(obj)	an array of the keys
Object.values(obj)	an array of the values
Object.entries(obj)	an array of [key, value] pairs

7 Nested objects

An object can hold another object inside it. You reach in with more dots.

```
const student = {  
  name: "Rahul",  
  address: { city: "Delhi", pin: 110001 }  
};  
student.address.city; // "Delhi"
```

8 Arrays (and arrays of objects)

An **array** stores multiple values in order. An **array of objects** is the shape real apps use most, a list of products, users, or orders.

```
const products = [  
  { id:1, name:"Laptop", price:65000, stock:true },  
  { id:2, name:"Mouse", price:500, stock:true },  
  { id:3, name:"Keyboard", price:1200, stock:false },  
  { id:4, name:"Monitor", price:18000, stock:true }  
];
```

Where this data comes from: in a real app you rarely type this list. It arrives from a **network call** to a server (an API). We opened shorterloop.com, watched the network request in DevTools, and saw the response come back as exactly this shape, an array of objects. These methods are how you then work with it.

forEach() - do something for every item

```
products.forEach(p => console.log(p.name));
```

Real world: reading out an attendance register, one name at a time. Returns nothing.

map() - build a NEW array

```
const names = products.map(p => p.name);
```

Real world: collect just the names of all students into a new list.

filter() - keep the matching items

```
const expensive = products.filter(p => p.price > 10000);
```

Real world: all students scoring above 90%.

find() - the first match only

```
const laptop = products.find(p => p.name === "Laptop");
```

Real world: find one student by roll number. Returns the item, not an array.

some() - is at least one true?

```
products.some(p => p.stock); // true if any in stock
```

Real world: is any classroom empty? Returns true or false.

every() - are they all true?

```
products.every(p => p.stock); // true only if ALL in stock
```

Real world: did everyone submit homework?

sort() - put them in order

```
products.sort((a, b) => a.price - b.price); // cheapest first
```

Real world: rank students by marks.

reduce() - boil it down to ONE value

```
const total = products.reduce((sum, p) => sum + p.price, 0);
```

Real world: add up a whole bill into a single total. The 0 is the starting value.

The big split: `map / filter` give you a new array. `find` gives one item. `some / every` give true or false. `reduce` gives one value. `forEach` gives nothing, it just runs.

★ quick reference

method	returns	use it for
forEach	nothing	run code on each item
map	a new array	transform every item
filter	a new array	keep items that match
find	one item	the first match
some	true / false	does any match?
every	true / false	do all match?
sort	the sorted array	put items in order
reduce	one value	combine into a total

9 Mini project: Student Management System

Take this list and answer four questions with four methods:

```
const students = [  
  { id:1, name:"Rahul", marks:92 },  
  { id:2, name:"Aman", marks:81 },  
  { id:3, name:"Riya", marks:96 }  
];  
  
// map    -> all the names  
// filter -> marks above 90  
// find   -> the student with id 2  
// reduce -> total marks of everyone
```

Try it before you look anything up. Each answer is one line. If you can do these four, you understand array methods.

10 Practice questions

1. Create Employee, Mobile and Car objects.
2. Add, update and delete properties on them.
3. Loop through an object's properties.
4. Create an array of 5 students.
5. Use map() to get their names.
6. Use filter() to get students above 80 marks.
7. Use find() to search for one student.
8. Use reduce() to calculate total marks.
9. Sort the students by marks.

interview questions

- What is an object?
 - Difference between an object and an array?
 - Difference between dot and bracket notation?
 - What is the **this** keyword?
 - Difference between map() and forEach()?
 - Difference between filter() and find()?
 - What does reduce() do?
 - When do you use some() and every()?
-

Class MOM

Module	Module 1 - JavaScript
Topics	Objects, and Array methods
Instructor	Dinesh Rawat

what we covered

- **Objects** - key-value pairs, why we use them, dot vs bracket access, add/update/delete
 - **Methods and this**, looping with for...in, and Object.keys / values / entries
 - **Nested objects** and reaching into them
 - **Arrays of objects**, the shape real API data comes in
 - the **array methods**: forEach, map, filter, find, some, every, sort, reduce
-

- saw real data live by watching a **network call on shorterloop.com** in DevTools
- a **mini project** (Student Management System) to tie it together

key takeaways

- ★ an object groups related data; an array is an ordered list
- ★ map and filter make new arrays; find returns one item; reduce returns one value
- ★ real app data is an array of objects, and these methods are how you use it

